comp10002
Foundations of Algorithms

Semester Two, 2025

Now What?

Lecture slides prepared by Alistair Moffat

Finishing up

ESS Survey

More computing?

Assessment

Exam overview

The hidden life of Artem and Alistair

Farewell to the spectators

End of Semester Survey

You have probably already been getting reminders about the official University end of semester survey.

Your feedback will guide efforts for further resources to be prepared next year, and help improve the subject.

Please spend the next two minutes doing the survey. Answer the "ratings" questions first; and then note one thing that went well, and one thing that could be improved. (And if nothing needs improvement, write "*algorithms were fun*").

Yes, find it and open it now, I'll wait two minutes!

More computing?

The most obvious next subject is comp20007 Design of Algorithms, offered in semester 1 each year.

It is the pathway through to further languages (Java, in swen20003) and technologies (databases, in info20003; data handling, in comp20008).

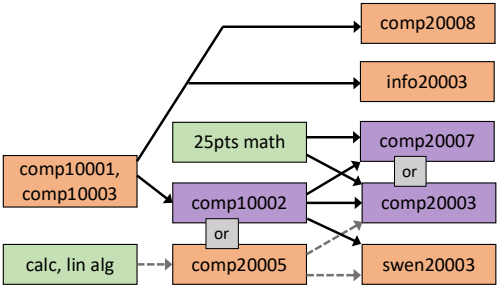
Going through comp20003 Algorithms and Data Structures (offered semester 2) is also an option, but has some overlap in content with comp10002. For some of you, that might be a *good* thing.

Subject comp20005 Intro. to Numerical Computation in C is restricted against comp10002. You can't take both.

More computing?

If you are planning a Maths, Physics, Bio, Geomatics, etc, major, computing skills are still critical.

Use your second- and third-year elective slots wisely. In the BCom too!



Want to combine two disciplines? Add the Diploma in Computing to your primary major. In the BCom too!

Assessment

Your final mark is the combination of five components.

Task	When	Marks
Weekly Homeworks (best 5 marks)	Weeks 2–9	5%
MST	Week 5	15%
Assignment 1	Week 8	20%
Assignment 2	Week 11	20%
Examination	November	40%

To pass the subject as a whole you must get 50 in total, and must also attain at least 22/55 (combined) in the MST plus exam, and 16/40 (combined) in the two assignments.

Hurdles

No one is prevented from sitting the exam.

But if your marks to date are already low, you should perhaps make a considered decision whether to focus on your other subjects (abandoning 1 and passing 3 might be better than attempting 4 and failing 2); or whether to try and stun us in the exam.

Decisions to fail students get made very carefully, based on the litmus abilities that are exposed via the exam questions.

Assessment

The exam will be very challenging! Not everyone will be able to answer all the questions, or even finish the ones they can.

Overall, we'll be aiming at the same final distribution as always – an overall average of ≈65%, and ≈20% H1, and will scale the exam (normally up) to get there if required.

Many students will already have reached 50 marks by the start of the exam – class median is currently approximately 30/40, and will be around 45/60 after Assignment 2 is marked. *The exam helps us decide which of students will go on to get over 85*!

The subject failure rate is typically ≈20%. Many of those people fail because they didn't complete assessment items.

The exam

The exam is based around four themes:

- Knowledge of C programming skills: calculation, selection, iteration, and abstraction;
- Knowledge of data types and of data structures in C: numbers, arrays, structs, linked structures (lists and trees), and combinations of them;
- Knowledge of the fundamentals of algorithm design and analysis, including asymptotic execution times, "good" versus "bad" algorithms; and
- Knowledge of algorithms for dictionaries and for sorting, and being able to balance their various attributes.

C control structures

Basic litmus tests:

- Integer number representations, general idea of floating point representations;
- Simple arithmetic and operations (function to sum a sequence);
- Loops and selection on scalars (prime numbers and etc);
- Use of pointer arguments in functions;
- Recursion.

C data structures

Basic litmus tests:

- Arrays, including functions that process arrays;
- Hierarchical construction of types and declarations (using constants, typedefs, structs, and arrays);
- Manipulation of structs, and arrays of structs, including in functions, and including via pointers;
- Principles of recursive structures using pointers: lists and trees.

Fundamentals of algorithms

Basic litmus tests:

- Functional growth rates and principles of analysis;
- General appreciation of dictionary data structures, including arrays, lists, and binary search trees;
- Binary and linear search;
- General appreciation of $O(n \log n)$-time sorting algorithms;
- Problem solving techniques and their various application domains.

To score well

Add in:

- Details of int and float number representations;
- Program correctness arguments, and formulation of loop invariants;
- Pattern matching and string search algorithms, string indexing data structures;
- Functions over recursive structures (lists and trees);
- Details of dictionary structures using arrays, trees, lists, and hashing;
- Details of quicksort, mergesort, heapsort, including analysis and relative merits;
- Priority queues, including implementation via a heap.

To score really well

There will be at least one question that will involve on-the-spot creativity or insight.

Maybe a problem we haven't discussed, or a related challenge that can be solved by adapting something you have been shown.

Maybe an analysis of a new algorithm, or some aspect of something covered only in passing in the lectures.

Maybe working backward from an analysis, to derive an algorithm that meets a given bound, drawing on your knowledge from different parts of the subject, in a "Students A, B, and C" scenario.

Exam preparation

- Start well in advance (now!);
- Lay out a study schedule that balances your exam timetable against particular subject needs;
- Put a copy of your exam timetable in a place where others will see it too;
- Scale back your social activities and other outside commitments;

Exam preparation

- Do (some of) your study with other people committed to passing;
- Read the whole book again starting at page 1 (or read it for the first time). Don't rely only on the lecture slides.
- Listen to (some of) the lectures again. They will almost certainly make more sense the second time round.

Exam preparation

- Revisit as many of the Ed exercises as you can manage, doing them from "blank screen" state all over again;
- Read the LMS site from top to bottom;
- Review the on-line workshop solutions;
- Do "heaps" of programming, making them all work.
- Eat well, sleep well, and make sure you get regular sunshine and exercise.

Exam technique

- Identify the key words in each question: write a function, or describe, and so on.
- Watch out for you should, you may, and you may not;
- If you cannot write the C code, write answers in English describing the approach you planned to use.
- Make sure that your web browser isn't submitting Chinese!
- Blank paper will be provided, to allow "working out" before you enter answers in the LMS Quiz. So bring a pen along.

Exam technique

- Be brief and to the point in any written answers;
- Ask yourself "what competency is being tested with this question?", then try and deliver;
- See if you can include a surprise at some stage – an extension, or insightful comment to make the marker smile, or even a new computing joke that can be added to the collection.

Exam technique

If you think you have found a mistake (it does happen!), use the exam's final zero-mark text answer box to write down the question number, your assumptions, and what the alternative assumptions might have been.

Then continue on the basis of your assumptions. (Plus make sure that you add in your answer to the box that question is actually for)

Don't get agitated about the problem, and don't just freeze and do nothing. We will assess your response to the unexpected situation, even though that situation may not have been planned.

Academic honesty

The exam is a *closed book assessment*. That means you must not:

- make use of any aids such as printed or softcopy notes;
- make use of electronic resources such as Grok or gcc;
- make use of web resources such as ChatGPT, Google, or Stackoverflow;
- communicate with any other student in regard to this assessment before you undertake it, while you are undertaking it, or after you have completed it.

Academic honesty

The only window you may have active on your computer during the examination is the LMS Quiz.

Be careful not to "accidentally" type or click any items during the reading time. The invigilators may regard that as Academic Misconduct.

Difficulties?

Be sure to try out the LMS "Practice Exam – Technical Check" Quiz item in the days before the exam.

The University IT Help will be on call at the exam venue if you encounter technical difficulties, and will also have spare "loan" computers available if yours fails at short notice.

Power will be available at every seat, so bring your charger too. Keyboards and mice may also be brought in.

Questions?

Yes, there is a Practice Exam, "Quizzes → Practice Exam" in the LMS. Sample solutions to Sections 3 and 4 will be released a week prior to the Exam. No spoilers before then please.

Assignment 2 marks will be released before the exam. Tentative goal for that is 3 November.

The FYC will be closed during swotvac, and then open again from 2–4pm between 3 and 14 November (but not on November 4).

You can post questions to the Discussion forum at any time, and we'll monitor that until a few hours prior to the exam.

The hidden life of Artem and Alistair

We are both T&R academics: T=40%, R=40%, S=20%.

T = "teaching".

What is S?

What is R?

The hidden life of Artem

Goal Recognition Using Off-The-Shelf Process Mining Techniques

Artem Polyvyanyy
The University of Melbourne
artem.polyvyanyy@unimelb.edu.au

Zihang Su
The University of Melbourne
zihangs@student.unimelb.edu.au

Nir Lipovetzky
The University of Melbourne
nir.lipovetzky@unimelb.edu.au

Sebastian Sardina
RMIT University
sebastian.sardina@rmit.edu.au

ABSTRACT

The problem of *probabilistic goal recognition* consists of automatically inferring a probability distribution over a range of possible goals of an autonomous agent based on the observations of its behavior. The state-of-the-art approaches for probabilistic goal recognition assume the full knowledge about the world the agent operates in and possible agent's operations in this world. In this paper, we propose a framework for solving the probabilistic goal recognition problem using process mining techniques for discovering models that describe the observed behavior and diagnosing deviations between the discovered models and observations. The framework imitates the principles of *observational learning*, one of the core mechanisms of social learning exhibited by humans, and relaxes the above assumptions. It has been implemented in a publicly available tool. The reported experimental results confirm the effectiveness and efficiency of the approach, both for rational and irrational agents' behaviors.

is executing, and a smart house ought to understand whether the household is trying to cook, watch a movie, or sleep by sensing her behavior (e.g., household entered the kitchen, turned on the light, and set the coffee machine timer). As the need for more autonomous systems increases, so does the attention on the GR problem [35], with numerous applications, from the support of adversarial reasoning for games and the military [20, 36] to smart homes and wheelchair movement [10, 34] to trajectory/manoeuvre prediction [12, 19, 21] and human-computer collaboration [22].

In traditional GR approaches, observations of agent's actions are "matched" to a plan (the one judged to being carried out by the agent) in a predefined library encoding the standard operational procedures of the domain [8, 11, 18]. While the first proposals did not accommodate uncertainty, numerous subsequent probabilistic solutions have later been developed [35]. Nonetheless, the challenge to obtain or hand-code the possible set of activities in a domain has

The hidden life of Artem

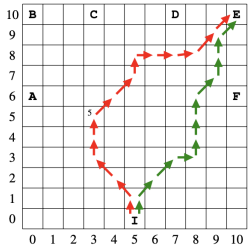


Figure 1: Two walks of an agent in a grid from initial cell I to goal cell E: rational (green) and irrational (red).

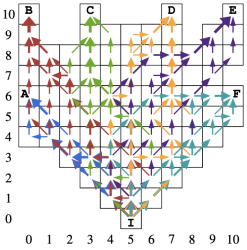


Figure 2: Observed agent behaviors from initial cell I to the six goal cells from Figure 1.

After explaining the approach, we report experimental results providing evidence of its effectiveness. In particular, we conducted goal recognition in three real-world domains from the Business Process Intelligence community *for which only event logs are available*, to infer a probability distribution over the goals. Intuitively, the more discrepancies between the observed behavior and a skill model, the less likely the agent is attempting to achieve the corresponding goal.

Figures 3(a) and 3(b) show probability distributions over the goals

The hidden life of Alistair

Large-Alphabet Semi-Static Entropy Coding Via Asymmetric Numeral Systems

ALISTAIR MOFFAT, The University of Melbourne
MATTHIAS PETRI, The University of Melbourne

An entropy coder takes as input a sequence of symbol identifiers over some specified alphabet and represents that sequence as a bitstring using as few bits as possible, typically assuming that the elements of the sequence are independent of each other. Previous entropy coding methods include the well-known Huffman and arithmetic approaches. Here we examine the newer asymmetric numeral systems (ANS) technique for entropy coding, and develop mechanisms that allow it to be efficiently used when the size of the source alphabet is large – thousands or millions of symbols. In particular, we examine different ways in which probability distributions over large alphabets can be approximated, and in doing so infer techniques that allow the ANS mechanism to be extended to support large-alphabet entropy coding. As well as providing a full description of ANS, we also present detailed experiments using several different types of input, including data streams arising as typical output from the modeling stages of text compression software; and compare the proposed ANS variants with Huffman and arithmetic coding baselines, measuring both compression effectiveness, and also encoding and decoding throughput. We demonstrate that in applications in which semi-static compression is appropriate, ANS-based coders can provide an excellent balance between compression effectiveness and speed, even when the alphabet is large.

CCS Concepts: •**Theory of computation** →**Data compression;** •**Information systems** →*Data compression;* Search index compression; •**Mathematics of computing** →Coding theory;

The hidden life of Alistair


```
5:  for i ← 0 to m − 1 do
6:      assert: sml ≤ state < sml · radix
7:      while state ≥ K · radix · c(σ_i) do
8:          buffer[b] ← state mod radix
9:          b ← b + 1
10:         state ← state div radix
11:     f ← state div c(σ_i)
12:     r ← state mod c(σ_i)
13:     state ← f · M + shuffle[base[σ_i] + r]
14: encode(buffer, b, state − sml)
15: encode(buffer, b, m)
16: return ⟨buffer, b⟩
```

```
5:  for i ← 0 to m − 1 do
6:      assert: sml ≤ state < sml · radix
7:      f ← state div M
8:      r ← state mod M
9:      σ_i ← symbol[r]
10:     state ← f · c(σ_i) + rank[r]
11:     while state < sml do
12:         b ← b − 1
13:         state ← state · radix + buffer[b]
14: assert: state = initial_state
15: return reverse(σ_0...m−1)
```

Fig. 4. Complete ANS encoding and decoding processes, including renormalization for *state* and incremental output (encoder) and input (decoder). The auxiliary function *encode*(*buffer*, *b*, *val*) uses a non-ANS mechanism (for example, binary for *state* − *sml*, and the Elias δ code for *m*) to append *val* to *buffer*[·] and update the output pointer *b*; similarly, *decode*(*buffer*, *b*) extracts and returns that value from the encoded message in *buffer*[·]. Note also that it is assumed that the decoder processes *buffer*[·] from right to left.

		Current *state*, counting from zero										
		0	1	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	10 ···
"a"	5/16	0	2	6	8	12	**16**	**18**	**22**	**24**	**28**	32 ···
"b"	4/16	1	3	7	9	**17**	**19**	**23**	**25**	33 ···		
"c"	3/16	4	13	15	**20**	**29**	**31**	36 ···				
"d"	2/16	5	10	**21**	**26**	37 ···						
"e"	2/16	11	14	**27**	**30**	43 ···						

Table 4. Revised state transition table $T(\cdot,\cdot)$, using a scaled frequency distribution with $c(\text{"a"}) = 5$, $c(\text{"b"}) = 4$, $c(\text{"c"}) = 3$, and $c(\text{"d"}) = c(\text{"e"}) = 2$, and a shuffled frame of size $M = 16$ in which the symbol ordering is

The hidden life of Artem and Alistair

An Entropic Relevance Measure for Stochastic Conformance Checking in Process Mining

Artem Polyvyanyy
The University of Melbourne
Email: artem.polyvyanyy@unimelb.edu.au

Alistair Moffat
The University of Melbourne
Email: ammoffat@unimelb.edu.au

Luciano García-Bañuelos
Tecnológico de Monterrey
Email: luciano.garcia@tec.mx

***Abstract*—Given an event log as a collection of recorded real-world process traces, process mining aims to automatically construct a process model that is both simple and provides a useful explanation of the traces. Conformance checking techniques are then employed to characterize and quantify commonalities and discrepancies between the log's traces and the candidate models. Recent approaches to conformance checking acknowledge that the elements being compared are inherently stochastic – for example, some traces occur frequently and others infrequently – and seek to incorporate this knowledge in their analyses.**

Here we present an *entropic relevance* measure for stochastic conformance checking, computed as the average number of bits required to compress each of the log's traces, based on the structure and information about relative likelihoods provided by the model. The measure penalizes traces from the event log not captured by the model and traces described by the model but absent in the event log, thus addressing both precision and recall quality criteria at the same time. We further show that entropic relevance is computable in time linear in the size of the log, and provide evaluation outcomes that demonstrate the feasibility of using the new approach in industrial settings.

are produced. For instance, most existing discovery algorithms strive to construct process models that fit, i.e., can replay, frequent traces from the input event logs. However, the vast majority of the discovery techniques construct non-deterministic models in which routing decisions are equiprobable, significantly limiting the ability of the models to explain the behaviors of the true processes sampled via the event logs. Indeed, if traces in a log suggest that failure was observed in nine out of ten traces, a model that says that (only) every second trace is erroneous is of reduced utility, and potentially harmful to subsequent decision-making activities. By accumulating event data over a long time, the log approaches the true event and trace frequencies. If this information about the true process were able to be reflected in the discovered model, the model would generate better process simulations, and hence better predictions of future processes.

We refer to process mining endeavors that process and produce artifacts with explicit information on the likelihood of

The hidden life of Artem and Alistair

precision and recall quality criteria. Such duality is indeed expected in the case of stochastic conformance checking. Finally, given a model and log, relevance is computable in time linear in the size of the log, making it – as we demonstrate below – useful when evaluating the quality of the process discovery algorithms commonly used in the industry.

The remainder of the paper proceeds as follows. The next section presents an overview of the compression methodology we employ, and describes how an event log can be encoded via a stochastic process model. Section III discusses formal stochastic models used in process mining. Based on these models, Section IV presents the notion of entropic relevance. Section V discusses the results of our evaluation of the entropic relevance measure, while Section VI summarizes related work.

II. Motivation and Overview

We employ a minimum description length compression-based framework to measure the quality of process models. Two key observations make this possible: that a good process model is one which accurately describes an observed set of traces, taken as a sample from an underlying universe of traces; and that the "describes" operation can be precisely quantified by assessing the cost of compressing the set of traces relative to the stochastic language expressed by the model. A signal benefit of this *entropic relevance* approach is that not only is the model *structure* an influence on its measured usefulness, but also the *probability* of each individual trace having emerged from the model. A relationship fundamental to information theory is critical to understanding the new approach: if a symbol e of probability $p(e)$ occurs, then the information conveyed by that occurrence is $-\log_2 p(e)$ bits,

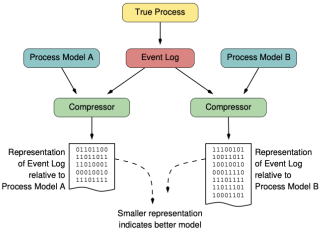


Fig. 1: To measure the entropic relevance of a process model to a collection of traces, the model's structure and probabilities are used to compute the cost in bits of losslessly representing the traces relative to the model model. Better models lead to a shorter compressed forms.

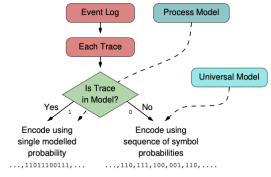


Fig. 2: The two possible options when encoding a set of traces with respect to a probabilistic model: each trace either has a non-zero

The hidden life of Artem and Alistair

arcs. Finally, the action and arc frequencies assigned by the corresponding frequency functions are encrypted next to the respective actions and arcs.

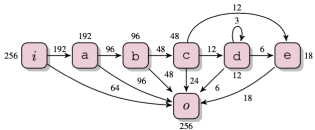


Fig. 5: An FDAG.

Van der Aalst [1] observes that practitioners can interpret FDAGs in different ways. Because of the filtering step, it is possible to associate a given FDAG with different collections of traces. He then discusses several pitfalls this phenomenon can lead to in practice. We agree with those observations, and, for our purpose, fix one such possible interpretation. To avoid ambiguities, next, we define our interpretation rigorously as a mapping from a given FDAG to the corresponding SDFA.

presumed to be encoded using the probability of t according to A, i.e., $L_A(t)$. Otherwise, t is presumed to be encoded using background knowledge about the log. These two modes are captured in this next definition.

Definition IV.1 (Trace compression cost):
Let $t \in E$ be a trace in an event log $E \in \mathcal{E}$, let $A \in \mathcal{A}$ be an SDFA, and let $bits : \Lambda^* \times \mathcal{E} \times \mathcal{A} \to \mathbb{R}^+$ be a function that maps any and every sequence $t \in \Lambda^*$ to the number of bits required to uniquely encode t, given a viable encoding that assumes knowledge of E and A. The *trace compression cost of t in the presence of E and A* is defined:

$$cost_{bits}(t, E, A) := \begin{cases} -\log_2(\mathcal{L}_A(t)) & t \in \hat{\mathcal{L}_A} \\ bits(t, E, A) & \text{otherwise.} \end{cases}$$

For simplicity, in this first presentation we use a straightforward background model to measure $bits(t, E, A)$, ignoring both E and A, and trivially encoding t by taking each individual action as an equi-probable symbol over the underlying alphabet augmented by an "end of string" symbol, and with each trace terminated by that special symbol: $bits(t, E, A) :=$

The hidden life of Artem and Alistair

Those three examples from 2020/2021 are at:

- `https://dl.acm.org/doi/abs/10.5555/3398761.3398886`
- `http://doi.org/10.1145/3397175`
- `http://doi.org/10.1109/ICPM49681.2020.00024`

And there's been plenty of others before and since!

- `http://www.polyvyanyy.com/publications.php`
- `https://people.eng.unimelb.edu.au/ammoffat/abstracts/`
- Look for other academics at `https://scholar.google.com/`.

Our international peers (and the University) judge us by what we invent. Plus the University judges our teaching and service.

Goodbye to the viewers at home

Nothing in the rest of the lecture is relevant to the exam.

It is simply some "academic entertainment" to reward the people who have been coming to lectures.

A1 and A2 signing off

Goodbye! Good luck with your other exams...

And remember us in the years to come as you have...

A1 and A2 signing off

Goodbye! Good luck with your other exams...

And remember us in the years to come as you have...

Algorithmic fun, **Fun, FUN!!!**